📘 Mini Project Requirement Document

Project Title: EventSphere – Event Management System

1️⃣ Project Overview

Students must build a **frontend Event Management Application** using:

- HTML5
- CSS3
- JavaScript (ES6+)
- ES6 Modules
- Classes (OOP)
- Async/Await
- JSON Server (Fake REST API)
- Form Validations

The application must follow **modular architecture and separation of concerns**.

2️⃣ Business Scenario

An organization conducts multiple events such as:

- Technical Workshops
- Cultural Programs
- Webinars
- Bootcamps
- Conferences

The system should allow:

- Admin to manage events
- Users to view and register for events

3️⃣ Functional Requirements

👨‍💼 Admin Module

Admin must be able to:

1. Add new event
2. Update existing event
3. Delete event
4. View all events
5. View number of registrations per event

👩‍💻 User Module

Users must be able to:

1. View available events
2. View event details
3. Register for an event
4. See confirmation after registration

4️⃣ Event Entity Structure

Students must design Event using a **Class**

Required properties:

```
{
  "id": number,
  "title": string,
  "description": string,
  "date": string,
  "location": string,
```

```
  "capacity": number,
  "availableSeats": number
}
```

5️⃣ Registration Entity Structure

Students must design Registration using a **Class**

```
{
  "id": number,
  "eventId": number,
  "participantName": string,
  "email": string,
  "phone": string
}
```

6️⃣ Technical Requirements

🔹 Must Use JavaScript Classes

- Event class
- Registration class
- Include:
    - Constructor
    - Validation method
    - Seat availability logic method

🔹 Must Use ES6 Features

Students must include:

- Arrow functions

- Template literals
- Destructuring
- Spread operator
- Default parameters
- Async/Await
- Modules (import/export)

🔹 **Must Use Async/Await**

All API calls must:

- Use Fetch API
- Use async/await
- Include try/catch error handling

🔹 **Must Use ES6 Modules**

Project must be divided into:

- models
- services
- controllers
- utils

Students must use:

export

import

And:

<script type="module">

7️⃣ **JSON Server Setup Requirements**

Students must:

1. Install JSON Server
2. Create db.json
3. Create two collections:
    - events
    - registrations
4. Run server on port 3000

8️⃣ Validations (Mandatory)

Students must implement:

Event Form Validation

- Title required
- Date cannot be past date
- Capacity must be positive number
- Location required

Registration Form Validation

- Name required
- Valid email format
- Phone must be 10 digits
- Prevent registration if no seats available

9️⃣ Business Rules

1. availableSeats must decrease after registration
2. Registration must not exceed capacity
3. Event cannot be deleted if registrations exist (bonus)

4. Show proper error messages
5. Show success notifications

🔟 UI Requirements

Students must:

- Use clean layout
- Use responsive design
- Use proper semantic HTML
- Avoid inline JS
- Use external JS files

Optional:

- Bootstrap allowed
- Loader animation for API calls

1️⃣1️⃣ Suggested Folder Structure

Students must follow:

```
event-management/
|
 ├── index.html
 ├── add-event.html
 ├── register.html
|
 ├── js/
|    ├── models/
|    ├── services/
```

```
|   ├── controllers/
|   ├── utils/
|   └── main.js
|
└── db.json
```

1️⃣2️⃣ API Endpoints Students Must Use

GET /events

POST /events

PUT /events/:id

DELETE /events/:id

GET /registrations

POST /registrations

1️⃣3️⃣ Non-Functional Requirements

- Code must be modular
- No global variable pollution
- Meaningful variable names
- Proper indentation
- Comments explaining logic
- Error handling must be implemented
- UI must not reload page after CRUD

1️⃣6️⃣ Deliverables

Students must submit:

1. Complete project folder
2. db.json file
3. Screenshot of working app
4. README file including:
    - Project description
    - How to run
    - Features implemented